

```

1 #!/usr/bin/env python3 -B
2 # ezr.py: explainable multi-objective optimization
3 # (c) 2026 Tim Menzies, timm@ieee.org, MIT license
4 """
5 Options:
6 --seed=1          random number seed
7 --p=2             distance (1,2=Man,Euclid)
8 --learn.leaf=3    examples per leaf
9 --learn.budget=50 rows to evaluate
10 --learn.check=5   guesses to check
11 --learn.start=4   initial labels
12 --bayes.m=2       m-estimate for Naive Bayes
13 --bayes.k=1       k-estimate (Laplace) for NB
14 --few=128         max unlabelled rows
15 --stats.cliffs=0.195 Cliff's Delta threshold
16 --stats.conf=1.36 KS test confidence
17 --stats.eps=0.35  margin of error multiplier
18 --show.show=30    tree display width
19 --show.decimals=2 decimal places for floats
20 --textmine.norm=0 CNB weight normalization
21 --textmine.yes=20 positive samples
22 --textmine.no=20  negative samples
23 --textmine.top=100 top TF-IDF features
24 --textmine.valid=20 repeats for stats testing
25 """
26 from __future__ import annotations
27 from time import perf_counter_ns as now
28 import os, re, random, sys, bisect, math
29 from pathlib import Path
30 from random import random as rand
31 from random import choices, choice, sample, shuffle
32 from math import log, log2, exp, sqrt, pi
33 from typing import Any, Iterable, Callable
34 from types import SimpleNamespace as S
35
36 # i:self j:iter n,m:int txt:str
37 # v:val w:weight x:indep y:dep
38
39 isa = isinstance
40
41 # T y p a s
42
43 type Qty = int|float
44 type Atom = str|bool|Qty
45 type Row = list[Atom]
46 type Rows = list[Row]
47 type Col = Num|Sym
48 type Cols = list[Col]
49 type Tree = Any
50 type Datas = list[Data]
51
52 # C o l u m n s
53
54 def Col(txt="", a=0):
55     """Num or Sym column based on name case."""
56     return (Num if txt[0].isupper() else Sym)(txt, a)
57
58 class Num:
59     """Summarizes a stream of numbers."""
60     def __init__(i, txt="", a=0):
61         i.txt, i.at, i.n = txt, a, 0
62         i.mu=i.m2=i.sd=0; i.heaven=txt[-1:]!="-"
63
64 class Sym:
65     """Summarizes a stream of symbols."""
66     def __init__(i, txt="", a=0):
67         i.txt, i.at, i.n, i.has = txt, a, 0, {}
68
69 def mid(col):
70     """Central tendency (mean or mode)."""
71     return col.mu if Num==type(col) else max(
72         col.has, key=col.has.get)
73
74 def spread(col):
75     """Variability (sd or entropy)."""
76     if Num==type(col): return col.sd
77     n = sum(col.has.values())
78     return -sum(v/n*log2(v/n) for v in col.has.values())
79
80 def norm(num, v):
81     """Normalize via logistic function."""
82     if v == "?": return v
83     z=max(-3,min(3,(v-num.mu)/(num.sd+1e-32)))
84     return 1/(1 + exp(-1.7*z))

```

```

91 #
92 # D a t a
93 #
94
95 class Data:
96     """Rows + summarized columns."""
97     def __init__(i, src=None):
98         src = iter(src or {})
99         i.rows,i._centroid = [],None
100         i.cols = Cols(next(src))
101         adds(src, i)
102
103 class Cols:
104     """Organize Num/Sym columns from headers."""
105     def __init__(i, names):
106         i.names = names
107         i.klass, i.xs, i.ys, i.all = None,[],[],[]
108         for j,txt in enumerate(names):
109             i.all.append(col := Col(txt,j))
110             if txt[-1] != "X":
111                 if txt[-1] == "!": i.klass = col
112                 role = i.ys if txt[-1] in "+-!" else i.xs
113                 role.append(col)
114
115 def clone(data, rows=None):
116     """Clone structure, optionally add rows."""
117     return adds(rows or [], Data([data.cols.names]))
118
119 def sub(it, v):
120     """Remove value/row (add with w=-1)."""
121     return add(it, v, -1)
122
123 def add(it, v, w=1):
124     """Add value/row to Data,Cols,Num,Sym."""
125     if Data is type(it):
126         it._centroid=None; add(it.cols,v,w)
127         if w>0: it.rows.append(v)
128         else : it.rows.remove(v)
129     elif Cols is type(it):
130         [add(col, v[col.at], w) for col in it.all]
131     elif v != "?":
132         if Sym == type(it):
133             it.has[v] = w + it.has.get(v, 0)
134         elif w < 0 and it.n <= 2:
135             it.n=it.mu=it.m2=it.sd=0
136         else:
137             it.n += w
138             delta = v - it.mu
139             it.mu += w * delta / it.n
140             it.m2 += w * delta * (v - it.mu)
141             it.sd=(max(0,it.m2)/(it.n-1))**.5 if it.n>1 else 0
142     return v
143
144 def mids(data):
145     """Centroid of all columns."""
146     data._centroid = data._centroid or [
147         mid(col) for col in data.cols.all]
148     return data._centroid
149
150 def adds(src, it=None):
151     """Add multiple items to target."""
152     it = it or Num()
153     [add(it, v) for v in (src or [])]
154     return it
155
156

```

```

156 #
157 # D i s t a n c e
158 #
159
160 def minkowski(items):
161     """Minkowski distance (param: the.p)."""
162     tot, n = 0, 1e-32
163     for item in items:
164         tot, n = tot + item**the.p, n + 1
165     return (tot/n) ** (1/the.p)
166
167 def disty(data, row):
168     """Distance to heaven on Y vars."""
169     return minkowski(
170         abs(norm(y,row[y.at])-y.heaven)
171         for y in data.cols.ys)
172
173 def distx(data, r1, r2):
174     """Distance between rows on X vars."""
175     return minkowski(
176         aha(x,r1[x.at],r2[x.at]) for x in data.cols.xs)
177
178 def aha(col, u, v):
179     """Distance between two values."""
180     if u == v == "?": return 1
181     if Sym == type(col): return u != v
182     u, v = norm(col, u), norm(col, v)
183     u = u if u!="?" else (0 if v>0.5 else 1)
184     v = v if v!="?" else (0 if u>0.5 else 1)
185     return abs(u - v)
186
187 def nearest(data, row, rows=None):
188     """Closest row on x-columns."""
189     return min(rows or data.rows,
190         key=lambda r2: distx(data, row, r2))
191
192 # B a y e s
193
194 def like(col, v, prior):
195     """How much a column likes a value."""
196     if type(col) == Sym:
197         return (col.has.get(v,0) +
198             the.bayes.k*prior)/(col.n+the.bayes.k)
199     sd=col.sd+1e-32; z=2*sd*sd
200     return exp(-(v-col.mu)**2/z)/sqrt(pi*z)
201
202 def likes(data, row, n_rows, n_klasses):
203     """Log likelihood of row given data."""
204     prior = (len(data.rows)+the.bayes.m
205         ) / (n_rows+the.bayes.m*n_klasses)
206     ls = [like(col,v,prior) for col in data.cols.xs
207         if (v:=row[col.at])!="?"]
208     return log(prior)+sum(log(v) for v in ls if v>0)
209
210 def classify(src, wait=10):
211     """Test then train: classify before update."""
212     src = iter(src)
213     h,cf,all = {}, None, Data([next(src)])
214     for n, row in enumerate(src):
215         want = row[all.cols.klass.at]
216         if n >= wait:
217             cf=dinc(want,max(h,key=lambda kl:likes(
218                 h[kl],row,len(all.rows),len(h))),cf)
219             if want not in h: h[want] = clone(all)
220             add(all, add(h[want], row))
221     return cf
222
223
224
225
226

```